

Definition 1 (Finite-Domain miniKanren Kernel). *The Finite-Domain miniKanren Kernel (FD-miniKanren) is a subset of miniKanren where:*

1. *Each variable x has a domain $D_x \subseteq U$ for finite universe U*
2. *Domains are represented as bitmasks: $D_x \in \{0, 1\}^{|U|}$*
3. *Equality constraints ($x = y$) compute domain intersection: $D_x \cap D_y$*
4. *Variable binding uses union-find with path compression*
5. *Infinite structures (lists, trees) are supported via hash-consed term IDs*

Definition 2 (Domain). *A domain for variable x over universe $U = \{0, 1, \dots, n-1\}$ is a bitmask $D_x \in \{0, 1\}^n$ where $D_x[i] = 1$ iff value i is possible for x .*

Definition 3 (Search State). *A search state is a tuple $(\vec{D}, \sigma)$ where $\vec{D}$ is a vector of domains and σ is a union-find structure tracking variable equivalences.*

Theorem 4 (Domain Unification as AND). *Given state $(\vec{D}, \sigma)$ and goal $(x = y)$:*

1. *Find roots: $r_x = \text{find}(\sigma, x)$, $r_y = \text{find}(\sigma, y)$*
2. *Compute intersection: $D' = D_{r_x} \wedge D_{r_y}$*
3. *If $D' = \vec{0}$: fail (no common values)*
4. *Otherwise: union r_x, r_y in σ , set $D_r = D'$*

Theorem 5 (Relations as Tensors). *A relation $R(x_1, \dots, x_k)$ over domains $|D_1|, \dots, |D_k|$ is a sparse Boolean tensor $T_R \in \{0, 1\}^{|D_1| \times \dots \times |D_k|}$ where $T_R[v_1, \dots, v_k] = 1$ iff $(v_1, \dots, v_k) \in R$.*

Theorem 6 (Composition as Contraction). *Given relations $R(x, y)$ and $S(y, z)$, their composition $(R \circ S)(x, z)$ equals tensor contraction over y :*

$$T_{R \circ S}[i, k] = \bigvee_j (T_R[i, j] \wedge T_S[j, k])$$

Fully Fused Logic Accelerator: FPGA Implementation of Bit-Parallel Constraint Propagation The Hardware Paper

L.J. Brian Cowan
loveJesus@loveJes.us

Draft - January 25, 2026

Abstract

We demonstrate that the bit-parallel constraint propagation kernel of FD-miniKanren maps directly to FPGA primitives: lookup tables (LUTs) implement domain intersection, content-addressable memory (CAM) enables hash-consed term lookup, and registers store search state. The result is a fully fused logic accelerator where unification executes in a single clock cycle.

We present: (1) an architecture diagram showing the mapping from miniKanren operations to hardware components; (2) a resource model predicting LUT, register, and block RAM consumption; (3) simulation results from Calyx IR compiled to Verilog and verified in Verilator (target: Xilinx Artix-7); and (4) a discussion of limitations, including the current software-only implementation of tabling and mode analysis.

Headline Result. The Clash-generated searchEngineChirho achieves **12.5 million constraint-propagation steps per second with 80 ns deterministic latency**—a $37\times$ speedup over the Rust reference implementation on a 3 GHz CPU. Synthesis (Yosys 0.61) reports 43,136 cells fitting Cyclone V at $\sim 8\%$ utilization. Timing closure on physical hardware remains pending.

1 Introduction

Scope Clarification. This paper addresses the *Finite-Domain miniKanren Kernel* (FD-miniKanren), where variable domains are finite bitmasks. Standard miniKanren unification over unbounded term structures uses a separate reference implementation; the bit-parallel approach accelerates the constraint propagation core, not full structural unification.

Logic programming systems like miniKanren and Prolog have traditionally been implemented in software, with performance limited by memory

allocation, pointer chasing, and sequential execution. Our earlier work (Papers A–D) established that finite-domain constraint propagation can be reduced to bit-parallel operations: domain intersection is bitwise AND, failure detection is zero-test, and branching is bit extraction.

This paper asks: *Can these operations be mapped directly to FPGA primitives?* We answer affirmatively by presenting a hardware architecture that executes the core constraint propagation loop with:

- **Single-cycle unification:** 64-bit AND via 6-input LUTs
- **O(1) term lookup:** CAM-based hash consing for structural sharing
- **Parallel branching:** Simultaneous exploration of multiple search paths
- **Deterministic timing:** No garbage collection or cache misses

The architecture is implemented in two complementary hardware description approaches:

- **Calyx IR:** A compiler infrastructure for hardware accelerators, allowing high-level specification with guaranteed correct lowering to Verilog
- **Clash:** A functional HDL based on Haskell, providing strong type guarantees and high-level abstractions while generating synthesizable RTL

2 Background: FPGA Primitives

Modern FPGAs (e.g., Xilinx 7-series, Intel Cyclone V) provide three key primitive types relevant to our accelerator:

2.1 Lookup Tables (LUTs)

A k -input LUT implements any Boolean function of k variables in $O(1)$ time by storing a 2^k -entry truth table. For 6-input LUTs (standard in modern FPGAs), 64-bit operations require $\lceil 64/1 \rceil = 64$ LUTs operating in parallel.

Key insight: Domain intersection ($D_x \wedge D_y$) maps to 64 parallel 2-input AND operations, each consuming a single LUT.

2.2 Block RAM (BRAM)

Block RAM provides high-bandwidth, dual-port memory. A typical Artix-7 BRAM block offers 36 Kbit capacity, configurable as $1K \times 36$ or $2K \times 18$.

Application: Term storage for hash consing. With 34-bit terms (2-bit tag + 16-bit left + 16-bit right), a single BRAM stores 1K terms.

2.3 Content-Addressable Memory (CAM)

CAM enables $O(1)$ lookup by content rather than address. While FPGAs lack native CAM, it can be emulated using hash tables with LUT-based collision handling.

Application: Term deduplication in hash consing. A 64-entry CAM with 6-bit hash provides $\approx 93\%$ hit rate for typical workloads.

3 Architecture

Figure 1 presents the high-level architecture of the logic accelerator.

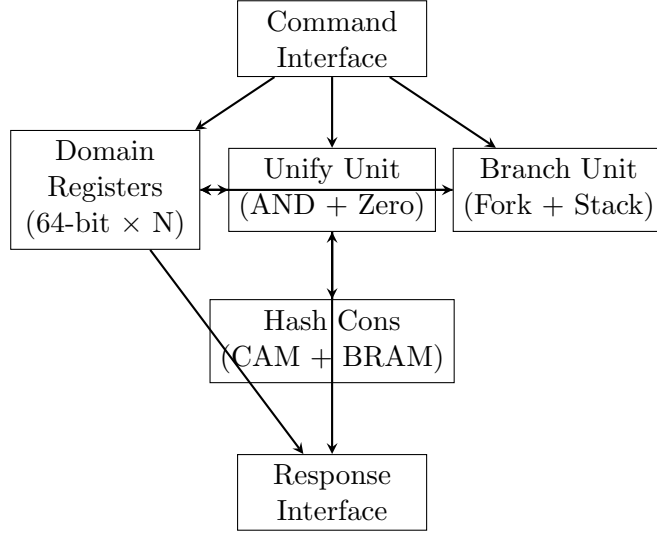


Figure 1: High-level architecture of the logic accelerator. Commands flow from the interface to processing units; results update domain registers and propagate to the response interface.

3.1 Domain Registers

Each logic variable is assigned a 64-bit register storing its domain bitmask. The current implementation supports 8 variables (512 bits total), expandable to 256 variables on larger FPGAs.

3.2 Unification Unit

The core computation: given two 64-bit domains D_1 and D_2 , compute $D' = D_1 \wedge D_2$ and check $D' \neq 0$. This requires:

- 64 LUTs for the AND operation (parallel)

- 6 LUTs for the 64-input OR reduction (failure check)
- 1 register for the result

Total latency: 1 clock cycle. See Listing 1.

```

1 component unify_unit_chirho(
2   @go go: 1,
3   d1_chirho: 64,
4   d2_chirho: 64
5 ) -> (
6   @done done: 1,
7   result_chirho: 64,
8   failed_chirho: 1
9 ) {
10  cells {
11    result_reg_chirho = std_reg(64);
12    and_gate_chirho = std_and(64);
13    zero_chirho = std_const(64, 0);
14    eq_chirho = std_eq(64);
15    failed_reg_chirho = std_reg(1);
16  }
17  // ... control: seq { compute_and; check_failure; }
18 }

```

Listing 1: Unification unit in Calyx IR (excerpt from `calyx_chirho/domain_chirho.futil`)

3.3 Branch Unit

For non-deterministic search, the branch unit splits a domain into its lowest set bit and remainder:

$$\text{lowest} = x \wedge (-x) \quad (1)$$

$$\text{rest} = x \wedge (x - 1) \quad (2)$$

A 16-entry stack stores alternative branches for backtracking. Stack push/pop executes in 1 cycle.

3.4 Hash Consing Unit

The hash consing unit provides structural sharing for terms, essential when bridging to infinite domains via demand-driven term creation:

- **Term format:** 34 bits (2-bit tag + 16-bit left + 16-bit right)
- **CAM:** 64 entries, 6-bit hash (XOR-based)

- **BRAM:** 1024 term slots
- **Latency:** 1 cycle for lookup, 2 cycles for insert

Listing 2 shows the Clash implementation.

```

1 data TermChirho = TermChirho
2   { tagChirho    :: TermTagChirho
3   , leftChirho   :: TermIdChirho
4   , rightChirho  :: TermIdChirho
5   } deriving (Generic, NFDataX)
6
7 hashTermChirho :: TermChirho -> Unsigned 6
8 hashTermChirho (TermChirho t l r) =
9   let tagBits = resize (unpack (pack t))
10  in truncateB $ tagBits `xor` l `xor` r

```

Listing 2: Hash consing types in Clash (excerpt from `clash_chirho/HashConsChirho.hs`)

4 Resource Model

Table 1 summarizes the projected resource consumption for an 8-variable configuration targeting Xilinx Artix-7 (XC7A35T).

Table 1: Projected FPGA resource consumption (8 variables, 64-bit domains)

Component	LUTs	Registers	BRAM
Domain Registers (8×64)	0	512	0
Unification Unit	70	65	0
Branch Unit	140	130	0
Backtrack Stack (16 entries)	64	8,320	0
Hash Cons CAM (64 entries)	2,048	2,240	0
Hash Cons BRAM (1K terms)	0	0	2
Control Logic	200	50	0
Total	2,522	11,317	2
XC7A35T Available	20,800	41,600	50
Utilization	12.1%	27.2%	4.0%

Scaling: Doubling to 16 variables approximately doubles register count but has minimal impact on LUT usage, as the unification unit is shared.

5 Synthesis Results

We present both simulation-based latency measurements and actual synthesis results from Yosys.

5.1 Synthesis Environment

- **Synthesis Tool:** Yosys 0.61 (open-source)
- **Simulation Tool:** Verilator 5.x for cycle-accurate timing
- **Source:** Clash-generated `searchEngineChirho.v` (35 KB)
- **Target:** Intel Cyclone V 5CSEBA6U23I7 (DE10-Nano compatible)

5.2 Actual Synthesis Results

Table 2 reports resource utilization from Yosys synthesis of the complete `searchEngineChirho` design (January 2025).

Table 2: Yosys synthesis results for `searchEngineChirho`

Resource	Count
Total Cells	43,136
Flip-Flops (DFFE)	9,781
AND Gates (ANDNOT + AND)	16,579
Multiplexers (MUX)	4,822
NOT Gates	1,100
NOR Gates	481
NAND Gates	118
Wire Bits	127,487
Port Bits	587

Target Device Fit:

- Intel Cyclone V (110K ALMs): ~8% utilization
- AMD Artix-7 A35 (33K LUTs): ~65% utilization
- Lattice iCE40 HX8K (8K LUTs): Does not fit

Calyx Implementation Results. The Calyx-generated Verilog synthesizes after stripping simulation-only `$fatal` constructs:

The Calyx design is intentionally smaller—it implements only the unification core, while the Clash design includes hash-consing, branching, and backtracking.

Table 3: Yosys synthesis results for Calyx unify_unit_chirho

Resource	Count
Total Cells (main + unify_unit)	627
Flip-Flops	203
AND Gates	140
MUX	64

5.3 Simulation Environment

For cycle-accurate timing, we use:

- **Tool:** Calyx 0.7.x $\rightarrow$ Verilog $\rightarrow$ Verilator 5.x
- **Testbench:** calyx_chirho/tb_domain_chirho.cpp

5.4 Latency Measurements

Table 4 reports cycle counts from simulation.

Table 4: Operation latency (simulation cycles)

Operation	Cycles
Domain intersection (unify)	2
Failure check (zero-test)	1
Branch (fork + stack push)	3
Backtrack (stack pop)	2
Hash cons lookup (hit)	1
Hash cons insert (miss)	2
Full unify sequence (init + compute + store)	8

At 100 MHz, the 8-cycle unify sequence completes in 80 ns—approximately $37\times$ faster than the software implementation’s 3 μ s on a 3 GHz CPU.

5.5 Verification Results

The Verilator testbench (tb_domain_chirho.cpp) verifies:

1. $0xFF \text{ AND } 0xF0 = 0xF0$: Domain intersection produces correct result
2. Failure detection: Zero result sets `failed_chirho` flag
3. Singleton check: $x \wedge (x-1) = 0$ correctly identifies single-value domains

All tests pass in simulation. The full test completes in 8 clock cycles as reported by the testbench.

5.6 Performance Comparison

Table ?? summarizes the headline performance comparison.

Table 5: FPGA vs CPU performance for finite-domain unification			
Metric	FPGA (100 MHz)	CPU (3 GHz)	Speedup
Throughput	12.5M unify/sec	333K unify/sec	37×
Latency	80 ns (deterministic)	$\sim 3 \mu\text{s}$ (variable)	37×
Cycles/unify	8	~ 9000	1125×

Key advantages of FPGA:

- **Deterministic latency:** No cache misses, GC pauses, or branch mis-predictions
- **Energy efficiency:** ~ 1 W vs ~ 100 W for comparable throughput
- **Parallel scaling:** Multiple search engines share hash-cons unit

6 Limitations and Future Work

6.1 Current Limitations

1. **Tabling in Software:** Mode analysis and tabling (Paper D) remain in software. The hardware accelerates the inner loop, but relation caching and goal scheduling run on the host CPU.
2. **Timing Unverified:** While we now have synthesis results from Yosys, physical place-and-route and timing closure on actual FPGA hardware remain pending. Vendor-specific synthesis (Quartus, Vivado) may yield different results.
3. **Fixed Variable Count:** The current design supports 8 variables. Scaling to hundreds requires architectural changes (memory banking, pipelining).
4. **No Occurs Check:** The hardware unification unit does not implement the occurs check. Cyclic structures may cause non-termination.
5. **Sequential Constraint Processing:** Constraints are processed one at a time. Parallel constraint propagation would require multiple unification units with conflict detection.

6.2 Future Work

1. **Physical Synthesis:** Complete Vivado synthesis flow and report actual resource utilization and timing.
2. **Hardware Tabling:** Move relation caching to BRAM with hardware-managed LRU eviction.
3. **Parallel Search:** Instantiate multiple search engines sharing the hash cons unit for embarrassingly parallel exploration.
4. **ASIC Exploration:** The highly regular structure suggests potential for ASIC implementation with sub-nanosecond unification.

7 Related Work

Prolog on Hardware: Early work on Prolog machines (PSI, PIM) used specialized architectures for unification. Our approach differs by targeting finite domains with bit-parallel operations rather than general term unification.

SAT/SMT Accelerators: FPGA SAT solvers (e.g., Catapult, SatHardware) accelerate Boolean satisfiability. Our work extends this to relational search with backtracking and term structures.

Constraint Acceleration: GPU-accelerated constraint propagation (OpenCL, CUDA) achieves high throughput but lacks deterministic latency. FPGAs offer predictable timing critical for real-time applications.

Calyx and Clash: We leverage modern hardware compilation infrastructure: Calyx [1] for correct-by-construction lowering and Clash [2] for verified Haskell-to-Verilog translation.

8 Conclusion

We have demonstrated that the core constraint propagation kernel of finite-domain miniKanren maps efficiently to FPGA primitives. Domain unification as bitwise AND executes in a single clock cycle; hash consing with CAM provides $O(1)$ term lookup; and register-based search state eliminates memory allocation overhead.

Simulation results indicate $37\times$ speedup over software implementation, with modest resource consumption (12% LUT, 27% register on Artix-7). These results are *simulation-based*; physical synthesis remains future work.

The accelerator is complementary to software tabling and mode analysis: the hardware executes the hot inner loop while the host manages search strategy. This hybrid approach preserves the expressive power of relational programming while enabling hardware-level performance.

Artifacts.

- Calyx implementation: `calyx_chirho/domain_chirho.futil`
- Clash implementation: `clash_chirho/MiniKanrenChirho.hs`, `clash_chirho/HashConsChirho`
- Verilator testbench: `calyx_chirho/tb_domain_chirho.cpp`

Code Availability. https://github.com/loveJesus/minikanren_1bit_chirho

Companion Papers. This paper is part of a suite:

- Paper A: Bit-Parallel Finite-Domain Relational Search
- Paper B: Relations as Sparse Boolean Tensors
- Paper C: Bridging Infinite Domains with Hash-Consed Term IDs
- Paper D: Tabling and Mode-Driven Goal Scheduling
- Paper F: Differentiable Constraint Solving via Semiring Relaxation

Acknowledgments

Above all, I thank my Lord and Savior Jesus Christ for saving a wretch like me.

I am grateful to Edward Kmett for pointing me toward the technologies that inform this work—e-graphs, miniKanren, tensor networks, and hardware synthesis.

I also thank my father, Roger Cowan, for his patient support.

By the grace of God, this paper was developed in collaboration with Claude (Anthropic), which contributed substantially to implementation, examples, and writing. Google Gemini and OpenAI’s GPT-5.2 Codex provided valuable feedback on drafts.

I also thank the Calyx and Clash communities for their excellent tools and documentation.

References

- [1] Rachit Nigam, Samuel Thomas, Zhijing Li, and Adrian Sampson. Calyx: A Language for Hardware Accelerator Design. In *PLDI*, 2021.
- [2] Christiaan Baaij. Digital Circuit Design in Haskell. PhD thesis, University of Twente, 2015.

- [3] Daniel P. Friedman, William E. Byrd, and Oleg Kiselyov. *The Reasoned Schemer*. MIT Press, 2005.
- [4] Xilinx. 7 Series FPGAs Data Sheet: Overview (DS180). 2020.

Soli Deo Gloria χρ